

第8章 异常处理结构与程序调试、测试

《Python程序设计，董付国》

异常处理结构与程序调试、测试

- 简单地说，异常是指程序运行时引发的错误，引发错误的原因有很多，例如除零、下标越界、文件不存在、网络异常、类型错误、名字错误、字典键错误、磁盘空间不足，等等。
- 如果这些错误得不到正确的处理将会导致程序终止运行，而合理地使用异常处理结构可以使程序更加健壮，具有更强的容错性，不会因为用户不小心的错误输入或其他运行时原因而造成程序终止。
- 也可以使用异常处理结构为用户提供更加友好的提示。
- 程序出现异常或错误之后是否能够调试程序并快速定位和解决存在的问题也是程序员综合水平和能力的重要体现方式之一。

8.1 基本概念

- 异常的常见表现形式

```
>>> x, y = 10, 5
```

```
>>> a = x / y
```

```
>>> A
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#35>", line 1, in <module>
```

```
    A
```

```
NameError: name 'A' is not defined
```

```
>>> 10 * (1/0)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#39>", line 1, in <module>
```

```
    10 * (1/0)
```

```
ZeroDivisionError: division by zero
```

8.1 基本概念

```
>>> 4 + spam*3
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in ?
```

```
NameError: name 'spam' is not defined
```

```
>>> '2' + 2
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in ?
```

```
TypeError: Can't convert 'int' object to str implicitly
```

```
>>> fp = open('123.data', 'rb')
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#2>", line 1, in <module>
```

```
    fp = open('123.data', 'rb')
```

```
FileNotFoundError: [Errno 2] No such file or directory: '123.data'
```

8.1 基本概念

函数用法不对，内置函数sorted()必须使用key参数指定排序规则

正确用法为sorted([111,22,3], key=str)

```
>>> sorted([111,22,3], str)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#89>", line 1, in <module>
```

```
    sorted([111,22,3], str)
```

```
TypeError: sorted expected 1 argument, got 2
```

函数用法不对，内置函数sorted()第一个参数必须是位置参数

```
>>> sorted(iterable=[111,22,3], key=str)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#93>", line 1, in <module>
```

```
    sorted(iterable=[111,22,3], key=str)
```

```
TypeError: sorted expected 1 argument, got 0
```

8.1 基本概念

列表是可变的，属于不可哈希对象，不能作为集合的元素

```
>>> data = {[1], [2]}
```

```
Traceback (most recent call last):
```

```
File "<pyshell#136>", line 1, in <module>
```

```
data = {[1], [2]}
```

```
TypeError: unhashable type: 'list'
```

大括号可以用来定义字典和集合，但不能同时包含“键:值”对和非“键:值”对

```
>>> data = {'a':97, 'b':98, 99, 100}
```

```
SyntaxError: invalid syntax
```

变量不存在，这样的情况一般是拼写错误造成的

```
>>> print(age)
```

```
Traceback (most recent call last):
```

```
File "<pyshell#141>", line 1, in <module>
```

```
print(age)
```

```
NameError: name 'age' is not defined
```

8.1 基本概念

```
# 文件不存在，这样的情况一般是路径错误或者拼写错误造成的
# 还有种可能是Windows操作系统隐藏了真正的扩展名，看到的扩展名并不是真的
# 使用字符串表示文件路径时，建议加字母r使用原始字符串，见2.1.3和8.1节
>>> with open('20200121.txt', encoding='utf8') as fp:
    content = fp.read()
```

```
Traceback (most recent call last):
```

```
File "<pyshell#144>", line 1, in <module>
```

```
    with open('20200121.txt', encoding='utf8') as fp:
```

```
FileNotFoundError: [Errno 2] No such file or directory: '20200121.txt'
```

8.1 基本概念

读取文本文件时使用了不正确的编码格式，见8.1节

```
>>> with open(r'C:/Python38/20200120.txt', encoding='utf8') as fp:  
    print(fp.read())
```

Traceback (most recent call last):

File "<pyshell#23>", line 2, in <module>

print(fp.read())

File "C:\Python38\lib\codecs.py", line 322, in decode

(result, consumed) = self._buffer_decode(data, self.errors, final)

UnicodeDecodeError: 'utf-8' codec can't decode byte 0xb6 in position 0:
invalid start byte

8.1 基本概念

以'w'模式打开的文件不能读取其中的内容，见8.1节

```
>>> with open('20200726.txt', 'w', encoding='utf8') as fp:  
    print(fp.read())
```

Traceback (most recent call last):

```
File "<pyshell#42>", line 2, in <module>  
    print(fp.read())
```

io.UnsupportedOperation: not readable

8.1 基本概念

```
>>> import sqlite3
>>> conn = sqlite3.connect('database.db')
>>> sql = 'SELECT * FROM student WHERE zhuanye="网络工程"'
# 数据库中不存在名为student的数据表
# 此时应检查SQL语句是否有拼写错误以及连接的数据库路径是否正确
>>> for row in conn.execute(sql):
    print(row)
```

```
Traceback (most recent call last):
  File "<pyshell#150>", line 1, in <module>
    for row in conn.execute(sql):
sqlite3.OperationalError: no such table: student
```

8.1 基本概念

输入的内容包含非数字字符，无法转换为整数

```
>>> number = int(input('请输入一个正整数: '))
```

请输入一个正整数: 12,345

Traceback (most recent call last):

File "<pyshell#152>", line 1, in <module>

number = int(input('请输入一个正整数: '))

ValueError: invalid literal for int() with base 10: '12,345'

```
>>> data = [1, 2, 3, 4, 5]
```

列表对象没有名为rindex的方法，无法调用

```
>>> data.rindex(3)
```

Traceback (most recent call last):

File "<pyshell#154>", line 1, in <module>

data.rindex(3)

AttributeError: 'list' object has no attribute 'rindex'

8.1 基本概念

Python不支持字符串与整数相加

```
>>> print('A' + 32)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#24>", line 1, in <module>
```

```
    print('A' + 32)
```

```
TypeError: can only concatenate str (not "int") to str
```

不支持对复数计算余数

```
>>> (3+4j) % (2+1j)
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#50>", line 1, in <module>
```

```
    (3+4j) % (2+1j)
```

```
TypeError: can't mod complex numbers.
```

8.1 基本概念

3(4+5)不能理解为3*(4+5)

这样写相当于把3当作函数来调用，出错并提示整数对象不可调用

```
>>> print(3(4+5))
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#28>", line 1, in <module>
```

```
    print(3(4+5))
```

```
TypeError: 'int' object is not callable
```

字符串漏掉了最后的引号，没有闭合

```
>>> print('Hello world)
```

```
SyntaxError: EOL while scanning string literal
```

8.1 基本概念

集合的大括号没有闭合

```
>>> print({3,4,5)
```

```
SyntaxError: closing parenthesis ')' does not match opening parenthesis  
'{'
```

在交互模式中每次只能执行一条语句

这样的错误一般是从文件中复制了多条语句到交互模式中执行造成的

```
>>> x = 3
```

```
y = 5
```

```
SyntaxError: multiple statements found while compiling a single  
statement
```

续行符“\”后面不能再有代码有效字符

```
>>> x = 3 + 5\ - 2
```

```
SyntaxError: unexpected character after line continuation character
```

8.1 基本概念

```
>>> from PIL import Image
>>> im = Image.open('1234.jpg')
# 调用方法时传递的实参数数量不对, 此时应使用help(im.getpixel)查看使用说明
# 这里正确的用法应该是print(im.getpixel((30,40)))
# 也就是使用表示横坐标和纵坐标位置的元组(30,40)作为方法getpixel()的参数
>>> print(im.getpixel(30,40))
Traceback (most recent call last):
  File "<pyshell#59>", line 1, in <module>
    print(im.getpixel(30,40))
TypeError: getpixel() takes 2 positional arguments but 3 were given
```

8.1 基本概念

- 语法错误和逻辑错误不属于异常，但有些语法错误往往会导致异常，例如由于大小写拼写错误而访问不存在的对象。
- 异常是指因为程序出错而在正常控制流以外采取的行为。当Python检测到一个错误时，解释器就会指出当前流已无法继续执行下去，这时候就出现了异常。
- 异常分为两个阶段：第一个阶段是引起异常发生的错误；第二个阶段是检测并处理阶段。
- 当程序出现错误，python会自动引发异常，也可以通过raise显式地引发异常。

8.1 基本概念

❖ 异常处理的作用

- ✓ 提高程序的健壮性和容错性
- ✓ 把晦涩难懂的错误提示转换为友好提示显示给最终用户

8.1 基本概念

- 既然这么神奇，那么是不是可以把所有代码都放到一个异常处理结构中呢？

try:

全部代码

except:

异常处理代码

X

8.1 基本概念

- 或者可不可以为每条语句都配上异常处理结构呢，反正也没啥坏处，不是吗？

```
...  
try:  
    print('ok')  
except:  
    pass  
...
```



8.1 基本概念

❖ 注意

- ✓ 不建议使用异常来代替常规的检查，如if...else判断。
- ✓ 应避免过多使用异常处理机制，只在确实需要时才使用。
- ✓ 捕捉异常时，应尽量精准，并针对不同类型的异常设计不同的处理代码。

8.2 Python内置异常类与自定义异常

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
    +-- StopIteration
    +-- ArithmeticError
    |   +-- FloatingPointError
    |   +-- OverflowError
    |   +-- ZeroDivisionError
    +-- AssertionError
    +-- AttributeError
    +-- BufferError
    +-- EOFError
    +-- ImportError
    +-- LookupError
    |   +-- IndexError
    |   +-- KeyError
    +-- MemoryError
    +-- NameError
    |   +-- UnboundLocalError
    +-- OSError
    |   +-- BlockingIOError
    |   +-- ChildProcessError
    |   +-- ConnectionError
    |   |   +-- BrokenPipeError
    |   |   +-- ConnectionAbortedError
    |   |   +-- ConnectionRefusedError
    |   |   +-- ConnectionResetError
    |   +-- FileExistsError
    |   +-- FileNotFoundError
    |   +-- InterruptedError
    |   +-- IsADirectoryError
    |   +-- NotADirectoryError
    |   +-- PermissionError
    |   +-- ProcessLookupError
    |   +-- TimeoutError
    +-- ReferenceError
    +-- RuntimeError
    |   +-- NotImplementedError
    +-- SyntaxError
    |   +-- IndentationError
    |   +-- TabError
    +-- SystemError
    +-- TypeError
    +-- ValueError
    |   +-- UnicodeError
    |   |   +-- UnicodeDecodeError
    |   |   +-- UnicodeEncodeError
    |   |   +-- UnicodeTranslateError
    +-- Warning
        +-- DeprecationWarning
        +-- PendingDeprecationWarning
        +-- RuntimeWarning
        +-- SyntaxWarning
        +-- UserWarning
        +-- FutureWarning
        +-- ImportWarning
        +-- UnicodeWarning
        +-- BytesWarning
        +-- ResourceWarning
```

8.2 Python内置异常类与自定义异常

- 可以继承Python内置异常类来实现自定义的异常类。

```
class ShortInputException(Exception):
    def __init__(self, length, atleast):
        Exception.__init__(self)
        self.length = length
        self.atleast = atleast

try:
    s = input('请输入 --> ')
    if len(s) < 3:
        raise ShortInputException(len(s), 3)
except EOFError:
    print('你输入了一个结束标记EOF')
except ShortInputException as x:
    print('ShortInputException: 长度是 %d, 至少应是 %d' % (x.length, x.atleast))
else:
    print ('没有异常发生.')
```

8.2 Python内置异常类与自定义异常

- 再例如

```
>>> class MyError(Exception):  
    def __init__(self, value):  
        self.value = value  
    def __str__(self):  
        return repr(self.value)  
  
>>> try:  
    raise MyError(2*2)  
except MyError as e:  
    print('My exception occurred, value:', e.value)
```

My exception occurred, value: 4

8.2 Python内置异常类与自定义异常

- 如果自己编写的某个模块需要抛出多个不同的异常，可以先创建一个基类，再创建多个派生类分别表示不同的异常。

```
class Error(Exception):  
    pass
```

```
class InputError(Error):  
    def __init__(self, expression, message):  
        self.expression = expression  
        self.message = message
```

```
class TransitionError(Error):  
    def __init__(self, previous, next, message):  
        self.previous = previous  
        self.next = next  
        self.message = message
```


8.3.1 try...except...

❖ **try**子句中的代码块放置可能出现异常的语句，**except**子句中的代码块处理异常。

try:

try块

被监控的语句

except Exception[as reason]:

except块

处理异常的语句

▪ 需要捕获所有异常时，可以使用异常类基类**BaseException**

try:

try块

except BaseException as e:

except块.....

不建议这样做

处理所有错误

8.3.1 try...except...

- 应用：要求用户必须输入数字字符串

```
>>> while True:
    x = input('Please input:')
    try:
        x = int(x)
        print('You have input {}'.format(x))
        break
    except Exception as e:
        print('Error.')
```

8.3.1 try...except...

- `except`子句可以在异常类名字后指定一个变量。

```
>>> try:
    raise Exception('spam', 'eggs')
except Exception as inst:
    print(type(inst))    # the exception instance
    print(inst.args)     # arguments stored in .args
    print(inst)          # __str__ allows args to be printed directly,
                        # but may be overridden in exception subclasses
    x, y = inst.args     # unpack args
    print('x =', x)
    print('y =', y)
```

```
<class 'Exception'>
('spam', 'eggs')
('spam', 'eggs')
x = spam
y = eggs
```

8.3.2 try...except...else...

- 如果try块内代码抛出了异常，就执行except块；如果try范围内没有就执行else块。

```
a_list = ['China', 'America', 'England', 'France']
```

```
while True:
```

```
    n = input('请输入字符串的序号')
```

```
    try:
```

```
        n = int(n)
```

```
        print(a_list[n])
```

```
    except:
```

```
        print('列表的下标必须为[0,3]之间的整数')
```

```
    else:
```

```
        break
```

8.3.2 try...except...else...

- 应用：查看多个文本文件分别有多少行

```
import sys
```

```
for arg in sys.argv[1:]:
```

```
    try:
```

```
        fp = open(arg, 'r')
```

```
    except FileNotFoundError:
```

```
        print('cannot open', arg)
```

```
    else:
```

```
        print(arg, 'has', len(fp.readlines()), 'lines')
```

```
        fp.close()
```

8.3.3 帶有多個except的try結構

❖ 帶有多個except的try結構

```
try:
```

try块

被监控的语句

```
except Exception1:
```

except 块1

处理异常1的语句

```
except Exception2:
```

except 块2

处理异常2的语句

```
[else:
```

没有异常时执行的代码]

8.3.3 带有多个except的try结构

- 例如

try:

```
x = input('请输入被除数: ')
```

```
y = input('请输入除数: ')
```

```
z = float(x) / float(y)
```

except ZeroDivisionError:

```
print('除数不能为零')
```

except TypeError:

```
print('被除数和除数应为数值类型')
```

except NameError:

```
print('变量不存在')
```

else:

```
print(x, '/', y, '=', z)
```

8.3.3 带有多个except的try结构

■ 再例如

```
import sys
try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
    f.close()
except OSError as err:
    print("OS error: {0}".format(err))
except ValueError:
    print("Could not convert data to an integer.")
except:
    print("Unexpected error:", sys.exc_info()[0])
    raise
```


8.3.3 带有多个except的try结构

- 将要捕获的异常写在一个元组中，可以使用一个except语句捕获多个异常：

```
import sys
try:
    f = open('myfile.txt')
    s = f.readline()
    i = int(s.strip())
    f.close()
except (OSError, ValueError, RuntimeError, NameError):
    pass
```

8.3.4 try...except...finally...

- 在该结构中，**finally**子句中的内容无论是否发生异常都会执行，常用来做一些清理工作以释放try子句中申请的资源。

try:

...

except 异常1:

...

except 异常2:

...

else:

...

finally:

...

无论如何都会执行

8.3.4 try...except...finally...

```
>>> try:  
    3 / 0  
except:  
    print(3)  
finally:  
    print(5)
```

3

5

8.3.4 try...except...finally...

- 使用异常处理结构保证文件总是能关闭

try:

```
f = open('test.txt', 'r')  
line = f.readline()  
print(line)
```

finally:

```
f.close()
```

❖ 但是，这样真的可以吗？

8.3.4 try...except...finally...

- 如果文件不存在，`finally`子句里的代码会抛出异常。

```
>>> try:
    f = open('test.txt', 'r')
    line = f.readline()
    print(line)
finally:
    f.close()
```

```
Traceback (most recent call last):
  File "<pyshell#17>", line 6, in <module>
    f.close()
NameError: name 'f' is not defined
```

8.3.4 try...except...finally...

- 如果try子句中的异常没有被处理，或者在except子句或else子句中出现了异常，那么这些异常将会在finally子句执行完后再次抛出。

```
>>> try:
    3 / 0
finally:
    print(5)
```

5

```
Traceback (most recent call last):
  File "<pyshell#52>", line 1, in <module>
    try:3/0
finally:
    print(5)
ZeroDivisionError: division by zero
```

8.3.4 try...except...finally...

- 例如，有函数定义如下：

```
>>> def divide(x, y):  
    try:  
        result = x / y  
    except ZeroDivisionError:  
        print("division by zero!")  
    else:  
        print("result is", result)  
    finally:  
        print("executing finally clause")
```

8.3.4 try...except...finally...

```
>>> divide(2, 1)
result is 2.0
executing finally clause
```

```
>>> divide(2, 0)
division by zero!
executing finally clause
```

```
>>> divide("2", "1")
executing finally clause
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in ?
```

```
  File "<stdin>", line 3, in divide
```

```
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```


8.4 断言与上下文管理

- 断言与上下文管理是两种比较特殊的异常处理方式，在形式上比异常处理结构要简单一些。

8.4.1 断言

❖ 断言语句的语法是：

```
assert expression[, reason]
```

- ✓ 当判断表达式`expression`为真时，什么都不做；如果表达式为假，则抛出异常。
- ✓ `assert`语句一般用于开发程序时对特定必须满足的条件进行验证，仅当`__debug__`为`True`时有效。当Python脚本以`-O`选项编译为字节码文件时，`assert`语句将被移除以提高运行速度。

8.4.1 断言

```
>>> a = 3
>>> b = 5
>>> assert a==b, 'a must be equal to b'
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    assert a==b, 'a must be equal to b'
AssertionError: a must be equal to b
>>> try:
    assert a==b, 'a must be equal to b'
except AssertionError as reason:
    print('%s:%s'%(reason.__class__.__name__, reason))
```

AssertionError:a must be equal to b

8.4.2 上下文管理语句

- 使用**with**自动关闭资源，可以在代码块执行完毕后还原进入该代码块时的现场。
- 不论何种原因跳出**with**块，不论是否发生异常，总能保证文件被正确关闭，资源被正确释放。
- **with**语句的语法如下：

```
with context_expr [as var]:  
    with块
```

8.4.2 上下文管理

- 应用：输出文本文件中的所有行。

```
with open("myfile.txt") as f:  
    for line in f:  
        print(line, end="")
```

8.5 使用pdb模块调试程序

- `pdb`是Python自带的交互式源代码调试模块，代码文件为`pdb.py`，但需要导入后才能使用其中的功能，使用该模块可以完成代码调试的绝大部分功能，包括设置/清除（条件）断点、启用/禁用断点、单步执行、查看栈帧、查看变量值、查看当前执行位置、列出源代码、执行任意Python代码或表达式等等。
- `pdb`还支持事后调试，可在程序控制下被调用。
- 可以通过`bdb`和`cmd`接口对该调试器进行扩展。

8.6 使用pdb模块调试程序

▪ pdb常用调试命令

完整/简写命令	用法示例	解释
a(rgs)		显示当前函数中的参数
b(reak) [[filename:]lineno function[, condition]]	b 173	在173行设置断点
	b function	在function函数第一条可执行语句位置设置断点
	b	不带参数则列出所有断点，包括每个断点的触发次数、当前忽略计数、以及与之关联的条件
	b 175, condition	设置条件断点，仅当condition的值为True时该断点有效
cl(ear) [filename:lineno bnumber [bnumber ...]]	cl	清除所有断点
	cl filename:lineno	删除指定文件指定行的所有断点
	cl 3 5 9	删除第3、5、9个断点

8.6 使用pdb模块调试程序

condition bnumber [condition]	condition 3 a<b	仅当a<b时3号断点有效
	condition 3	将3号断点设置为无条件断点
continue		继续运行至下一个断点或脚本结束
disable [bnumber [bnumber ...]]	disable 3 5	禁用第3、5个断点，禁用后断点仍存在，可以再次被启用
d(own)		在栈跟踪器中向下移动一个栈帧
enable [bnumber [bnumber ...]]	enable n	启用第n个断点
h(elp) [command]		查看pdb帮助
ignore bnumber [count]		为断点设置忽略计数，count默认值为0。若某断点的忽略计数不为0，则每次触发时自动减1，当忽略计数为0时该断点处于活动状态。

8.6 使用pdb模块调试程序

j(ump)	j 20	跳至第20行继续运行
l(ist) [first [,last]]	l	列出脚本清单，默认11行
	l m, n	列出从第m行到第n之间的脚本代码
	l m	列出从第m行开始的11行代码
n(ext)		执行下一条语句，遇到函数时不进入其内部
p(rint)	p i	打印变量i的值
q(uit)		退出pdb调试环境
r(eturn)		一直运行至当前函数返回
tbreak		设置临时断点，该类型断点只被中断一次，触发后该断点自动删除
s(step)		执行下一条语句，遇到函数时进入其内部
u(p)		在栈跟踪器中向上移动一个栈帧
w(here)		查看当前栈帧
[!]statement		执行语句，!与要执行的语句之间不需要空格，任何非pdb命令都被解释为Python语句并执行，可以调用函数或修改当前上下文中变量的值。
		直接回车则默认执行上一个命令，但如果上一个命令是list，则会列出接下来的11行代码

8.6 使用Jupyter Notebook调试程序

The screenshot displays a Jupyter Notebook environment with two tabs: 'example.ipynb' and 'scientific.ipynb'. The active tab, 'scientific.ipynb', contains a code cell with the following Python code:

```
In 15: 1 colors = np.random.rand(N)
        2 area = np.pi * (15 * np.random.rand(N))**2 # 0 to 15 point radii
        3 plt.scatter(x, y, s=area, c=colors, alpha=0.5)
        4 ?plt.scatter
        5 plt.show()
```

Below the code cell, a scatter plot is displayed, showing a distribution of points with varying sizes and colors (ranging from purple to yellow) on a coordinate system with x and y axes ranging from 0.6 to 1.0.

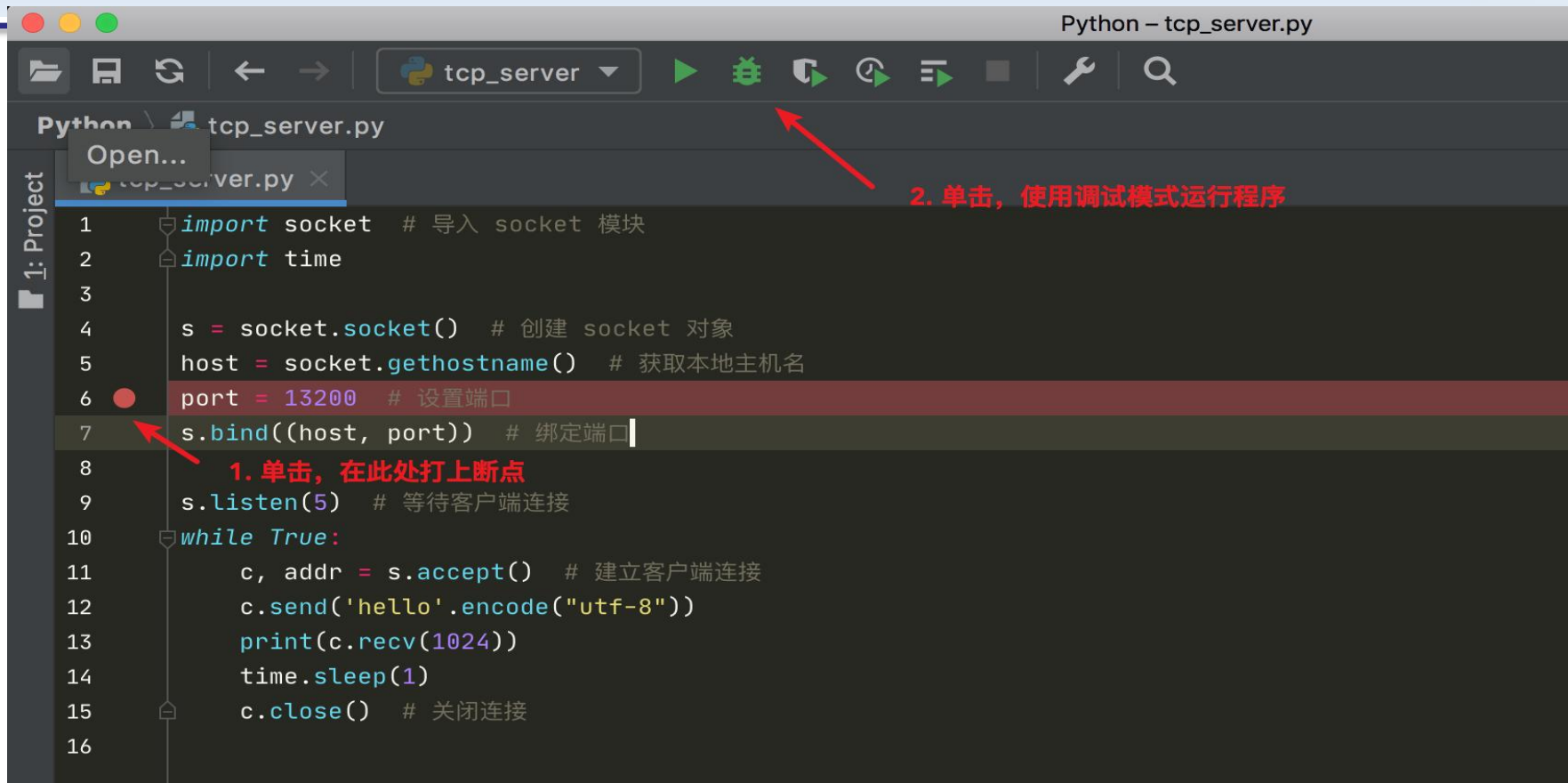
Below the plot, the Jupyter interface shows the 'Introspection' panel for the `plt.scatter` function. The panel displays the following information:

- Server: /Users/jetbrains/DataspellProjects/dsProject
- Introspection x
- Signature: `plt.scatter(x, y, s=None, c=None, **kwargs)`
- Docstring: A scatter plot of `*y*` vs. `**x*` with varying marker size and/or color.
- Parameters:
 - `x, y`: float or array-like, shape (n,)
The data positions.
 - `s`: float or array-like, shape (n,), optional
The marker size in points**2.
Default is ```rcParams['lines.markersize'] ** 2```.

The bottom status bar of the Jupyter Notebook shows the following information:

- Version Control
- Debug
- TODO
- Problems
- Terminal
- R Jobs
- Python Packages
- R Console
- Python Console
- Jupyter Server started at <http://localhost:8888> // Open in Browser (12.07.22, 16:27)
- 8:1
- LF
- UTF-8
- 4 spaces
- Python 3.9 (base)

8.6 使用PyCharm调试程序



8.6 使用VS Code调试程序

The screenshot shows the VS Code Python Debug Console interface. The top bar indicates the current environment is 'Python: Flask'. The left sidebar contains several panels: 'VARIABLES' (showing 'Locals' with 'name: 'Sebastian'' and 'today: datetime.date(2020, 3, 9)' with its attributes), 'WATCH' (showing 'name: 'Sebastian''), 'CALL STACK' (showing 'MainThread' and 'Thread-6'), and 'BREAKPOINTS' (showing 'app.py' with a breakpoint at line 9). The main editor shows the code for 'app.py', with a breakpoint set at line 9. A tooltip for the 'today' variable is visible, showing its attributes: 'day: 9', 'max: datetime.date(9999, 12, 31)', 'min: datetime.date(1, 1, 1)', 'month: 3', 'resolution: datetime.timedelta(days=1)', and 'year: 2020'. The bottom panel shows the 'DEBUG CONSOLE' with the output of the Flask application, including the command to run the app and the resulting output: 'Serving Flask app "app.py"', 'Environment: development', 'Debug mode: off', and 'Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)'.

Breakpoint

List of all available variables in the current scope

You can define some variables that you want to watch all the time

If you hover your mouse over a variable, you will see its current value

You can execute any Python code in the DEBUG CONSOLE tab

on 3.8.1 64-bit ('my-project': venv) 0 0 Python: Flask (flask-app) Ln 6, Col 12 Spaces: 4 UTF-8 LF Python

8.8 Python单元测试

- 软件测试对于保证软件质量非常重要，尤其是系统升级过程中对代码的改动不应该影响原有功能，是未来重构代码的信心保证。
- 几乎所有软件公司都有专门的测试团队来保证软件质量，但作为程序员，首先应该保证自己编写的代码准确无误地实现了预定功能，单元测试是保证模块质量的重要手段之一。

8.7 Python单元测试

- 从软件工程角度来讲，软件测试分为白盒测试和黑盒测试。
- ✓ **白盒测试**主要通过阅读程序源代码来判断是否符合功能要求，对于复杂的业务逻辑白盒测试难度非常大，一般以黑盒测试为主，白盒测试为辅。
- ✓ **黑盒测试**不关心模块的内部实现方式，只关心其功能是否正确，通过精心设计一些测试用例检验模块的输入和输出是否正确来判断其是否符合预定的功能要求。

8.7 Python单元测试

方法名称	功能说明	方法名称	功能说明
<code>assertEqual(a, b)</code>	<code>a == b</code>	<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>bool(x) is True</code>	<code>assertFalse(x)</code>	<code>bool(x) is False</code>
<code>assertIs(a, b)</code>	<code>a is b</code>	<code>assertIsNot(a, b)</code>	<code>a is not b</code>
<code>assertIsNone(x)</code>	<code>x is None</code>	<code>assertIsNotNone(x)</code>	<code>x is not None</code>
<code>assertIn(a, b)</code>	<code>a in b</code>	<code>assertNotIn(a, b)</code>	<code>a not in b</code>
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>	<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>
<code>assertAlmostEqual(a, b)</code>	<code>round(a-b, 7) == 0</code>	<code>assertNotAlmostEqual(a, b)</code>	<code>round(a-b, 7) != 0</code>
<code>assertGreater(a, b)</code>	<code>a > b</code>	<code>assertGreaterEqual(a, b)</code>	<code>a >= b</code>
<code>assertLess(a, b)</code>	<code>a < b</code>	<code>assertLessEqual(a, b)</code>	<code>a <= b</code>
<code>assertRegex(s, r)</code>	<code>r.search(s)</code>	<code>assertNotRegex(s, r)</code>	<code>not r.search(s)</code>
<code>setUp()</code>	每项测试开始之前自动调用该函数	<code>tearDown()</code>	每项测试完成之后自动调用该函数

8.9 性能测试

- 运行时间测试

```
import time
```

```
def demo():  
    start = time.time()  
    for i in range(9999999):  
        1+1  
    end = time.time()  
    print(end-start)
```

```
demo()
```


8.9 性能测试

- 内存占用测试。使用pip安装Python扩展库memory_profiler，然后编写下面的代码：

```
from memory_profiler import profile
```

```
@profile
```

```
def memory_test():
```

```
    test = []
```

```
    test.append([8]*100000)
```

```
    test.append([8]*200000)
```

```
    test.append([8]*300000)
```

```
    test.append([8]*400000)
```

```
    test.append([8]*500000)
```

```
memory_test()
```

8.9 性能测试

❖ 运行结果:

Line #	Mem usage	Increment	Line Contents
=====			
3	44.6 MiB	44.6 MiB	@profile
4			def memory_test():
5	44.6 MiB	0.0 MiB	test = []
6	45.4 MiB	0.8 MiB	test.append([8]*100000)
7	46.9 MiB	1.5 MiB	test.append([8]*200000)
8	49.2 MiB	2.3 MiB	test.append([8]*300000)
9	52.3 MiB	3.1 MiB	test.append([8]*400000)
10	56.1 MiB	3.8 MiB	test.append([8]*500000)